

PRACTICAL : WEEK 8

Memory Management and Copy-on-Write

Name	Sathish kumar
Roll Number	2520090008

Task 1: Dynamic Memory Allocation

Task: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Code / Execution

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p = malloc(5 * sizeof(int));
    int *q = calloc(5, sizeof(int));

    if (p == NULL || q == NULL)
        return 1;

    for (int i = 0; i < 5; i++)
        p[i] = i + 1;

    p = realloc(p, 10 * sizeof(int));

    if (p == NULL)
    {
        free(q);
        return 1;
    }

    for (int i = 5; i < 10; i++)
        p[i] = i + 1;

    for (int i = 0; i < 10; i++)
        printf("%d ", p[i]);

    printf("\n");

    free(p);
    free(q);

    return 0;
}
```

Task 1 – C program using malloc(), calloc(), realloc(), and free()

Output

```
pranay@LAPTOP-OP7R42IK:~$ nano memoryalloc.c
pranay@LAPTOP-OP7R42IK:~$ gcc memoryalloc.c -o memoryalloc
pranay@LAPTOP-OP7R42IK:~$ ./memoryalloc
1 2 3 4 5 6 7 8 9 10
pranay@LAPTOP-OP7R42IK:~$ |
```

Task 1 – Compilation and program execution output

```
pranay@LAPTOP-OP7R42IK:~$ valgrind --leak-check=full ./memoryalloc
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.26.0-0ubuntu1
See 'snap info valgrind' for additional versions.
pranay@LAPTOP-OP7R42IK:~$ |
```

Task 1 – Valgrind command result: Valgrind is not installed

```
pranay@LAPTOP-OP7R42IK:~$ valgrind --leak-check=full ./memoryalloc
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.26.0-0ubuntu1
See 'snap info valgrind' for additional versions.
pranay@LAPTOP-OP7R42IK:~$ |
```

Task 1 – Valgrind command result (additional screenshot)

Task 2: Copy-on-Write using fork()

Task: Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Code

```
pranay@LAPTOP-OP7R42IK:~$ nano cow.c
GNU nano 8.7.1 cow.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int *data = malloc(100 * sizeof(int));

    if (data == NULL)
        return 1;

    data[0] = 10;

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child before: %d\n", data[0]);

        data[0] = 999;

        printf("Child after: %d\n", data[0]);

        free(data);
    }
    else
    {
        wait(NULL);

        printf("Parent: %d\n", data[0]);

        free(data);
    }

    return 0;
}
```

Task 2 – C program demonstrating Copy-on-Write using fork()

Output

Task 2 – Program execution output

```
pranay@LAPTOP-OP7R42IK:~$ nano cow.c
pranay@LAPTOP-OP7R42IK:~$ gcc cow.c -o cow
pranay@LAPTOP-OP7R42IK:~$ ./cow
Child before: 10
Child after: 999
Parent: 10
pranay@LAPTOP-OP7R42IK:~$ |
```